

Rings: A peer-to-peer network for the sovereign age

Ryan J. Kung
ryankung@ieee.org

Created: February 8, 2023
Source: RingsNetwork/whitepaper repository
Timestamp: 2023-02-08T13:27:55Z
Last updated: July 4, 2026

Abstract

Rings is the specification

$$\mathcal{R} \setminus \setminus f = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathbb{T}_A^\varepsilon, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X, H) &= (X, \text{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \rightarrow \mathbb{T}_{A_\nu}^{\varepsilon_\nu}(S_\nu \times O_\nu). \\ \{D, V, M, H, Q, W\} &\subset \text{Obj}(\mathbf{Set}/R), \quad \text{CryptoCarrier} \cap R = \emptyset. \end{aligned}$$

1 Formal Model

Assumption 1 (System scope).

$$\begin{aligned} \text{fault} &= \text{fail-stop}, \\ \text{auth}(m, \sigma, k) &= V(k, \sigma, m), \\ \text{net} &= \text{async}, \\ \text{io} &= I_X : A_X^* \rightarrow \text{IO}, \\ \text{consensus} &\notin \text{Overlay}, \\ \text{order} &= \text{SidecarWitness}, \\ \text{crypto_ops} \cap \text{route_ops} &= \emptyset. \end{aligned}$$

Definition 1 (Identifier carrier).

$$\begin{aligned} M &= 2^{160}, \\ R &= \mathbb{Z}/M\mathbb{Z}, \\ \delta_a(x) &= (x - a) \bmod M, \\ (a, b]_R &= \{x \in R : 0 < \delta_a(x) \leq \delta_a(b)\}, \\ \text{RouteOps}_R &= \{0, +, -, \delta_a, (_, _)]_R\}, \\ \text{Mult}_R &\notin \text{RouteOps}_R. \end{aligned}$$

Definition 2 (Live topology).

$$\begin{aligned} N &\subset R, \quad N \neq \emptyset, \\ \text{owner}_N(k) &= \arg \min_{u \in N} \delta_k(u), \\ \text{pred}_N(n) &= \arg \min_{p \in N \setminus \{n\}} \delta_p(n), \\ \text{Succ}_N^q(n) &= \text{take}_q(\text{sort}_{\delta_n}(N \setminus \{n\})), \\ \delta_a|_N &= \text{injective}. \end{aligned}$$

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned}
r &= |\text{Succ}_N^r(n)|, \\
\text{Steady}(N) &\Rightarrow |N| \geq r + 1, \\
|N| < r + 1 &\Rightarrow \text{Init}(N), \\
\text{Maintain} &= \{\text{join}, \text{rectify}\} \\
&\quad \cup \{\text{stabilize}\}_{\text{CorrectChord}}.
\end{aligned}$$

[1]

Definition 3 (Replica set).

$$\text{Rep}_N^r(k) = \{\text{owner}_N(k)\} \cup \text{set}(\text{Succ}_N^{r-1}(\text{owner}_N(k))).$$

Definition 4 (Rings overlay object).

$$\mathcal{R}_N = (R, N, \text{owner}_N, \text{pred}_N, \text{Succ}_N^r, \text{Rep}_N^r, F, E),$$

$$\begin{aligned}
F &\subseteq N \times \mathbb{N} \times N, \\
E &= \prod_{v \in \text{VID}} E_v, \\
\text{SafetyRoot}(\mathcal{R}_N) &= (\text{pred}_N, \text{Succ}_N^r), \\
\text{PerfWitness}(\mathcal{R}_N) &= F.
\end{aligned}$$

Proposition 1 (Rotation equivariance).

$$\begin{aligned}
t \in R, \quad N + t = \{n + t : n \in N\} &\Rightarrow \\
\text{owner}_{N+t}(k + t) &= \text{owner}_N(k) + t.
\end{aligned}$$

Proof.

$$\begin{aligned}
\forall n \in N. \quad \delta_{k+t}(n + t) &= ((n + t) - (k + t)) \bmod M \\
&= \delta_k(n).
\end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned}
\text{PlacementOps} &= \{0, +, -, \delta_a, (a, b)_R\} \\
&\quad \cup \{\text{owner}_N, \text{Rep}_N^r\}, \\
\text{CryptoOps} &= \{\times, ^{-1}, \cdot_{\text{scalar}}, \text{interp}\} \\
&\quad \cup \{\text{pair}, \text{subgroup}\}, \\
\text{PlacementOps} \cap \text{CryptoOps} &= \emptyset, \\
\text{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}.
\end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID placement	$H_\nu : X_\nu \rightarrow R$ then owner map
Storage merge	join semilattice or explicit finalizer
E2E encryption	finite field or curve group
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table 1: Carrier boundaries in the Rings model

2 Architecture As Interfaces

Definition 5 (Layer tuple).

$$\begin{aligned}
\mathcal{A} &= (I, T, O, X, P, L) \\
I &= (\text{DID}, \Pi, \text{Session}) \\
T &= (C, A_T, \text{offer}, \text{answer}, \text{send}, \text{recv}) \\
O &= \mathcal{R}_N \\
X &= (S_X, A_X, \text{cap}, I_X) \\
P &= \{\mathcal{P}_\nu\}_{\nu \in \text{Namespace}} \\
L &= \text{App} \circ P
\end{aligned}$$

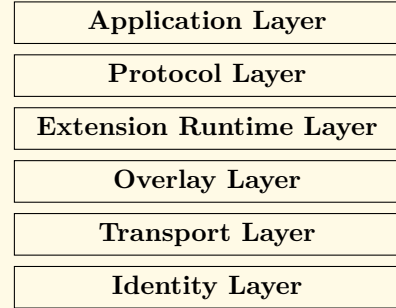


Figure 1: Layers of Rings Network

Definition 6 (Identity proof).

$$\begin{aligned}
\Pi &= (K, \Sigma, V, D, \text{enc}, \text{ttl}) \\
V &: K \times \Sigma \times B^* \rightarrow \{\top, \perp\}, \\
D &: K \rightarrow R, \\
\text{enc} &: \Sigma \rightarrow B^*, \\
\text{ttl} &: \Sigma \rightarrow \text{Time}.
\end{aligned}$$

Invariant 1 (Session authority).

$$d \in X_D, \quad s \in K.$$

$$\text{Session}(d, s, e) \Rightarrow V(k_d, \sigma_{d \rightarrow s}, d \| s \| e) = \top \wedge \text{now} < e.$$

$$\text{HopCheck}(d, s, e) \equiv \text{Session}(d, s, e).$$

Definition 7 (Transport interface).

$$T = (C, \text{offer}, \text{answer}, \text{send}, \text{recv})$$

$$A_T = \{\text{Offer}, \text{Answer}, \text{Ice}, \text{Open}, \text{Send}, \text{Recv}, \text{Close}\}.$$

$$I_T : A_T^* \rightarrow \text{IO}_{WebRTC} \cup \text{IO}_{Native}.$$

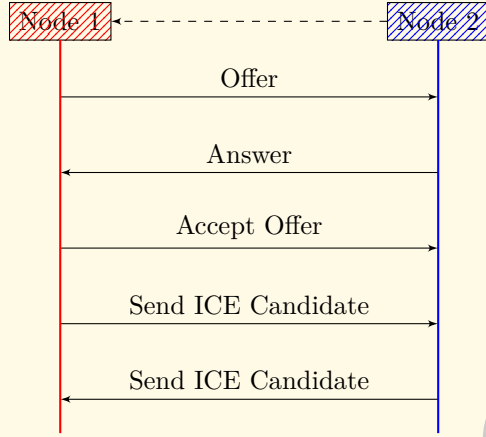


Figure 2: Standard SDP/ICE exchange

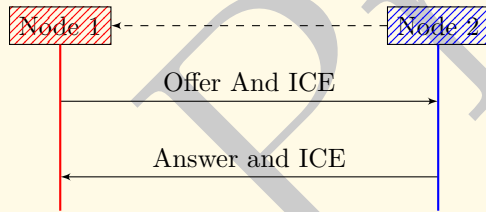


Figure 3: Rings single round-trip SDP/ICE exchange

Definition 8 (Runtime interpreter).

$$\mathcal{X} = (S_X, A_X, \text{cap}, I_X)$$

$$\text{cap} \subseteq A_X,$$

$$I_X : A_X^* \rightarrow \text{IO},$$

$$I_X(\epsilon) = \text{Noop},$$

$$I_X(\alpha \cdot \beta) = I_X(\alpha); I_X(\beta).$$

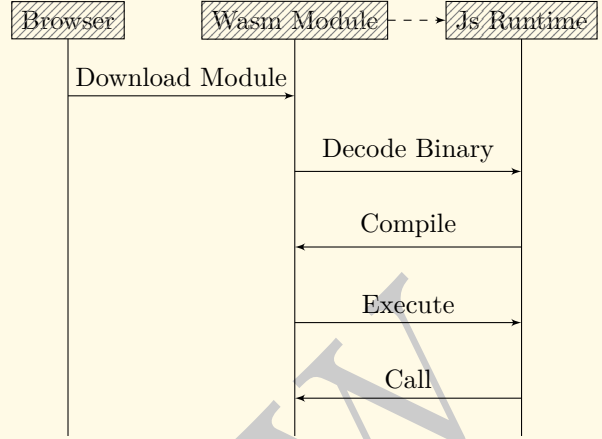


Figure 4: Wasm module execution boundary

Obligation 1 (Runtime confinement).

$$\begin{aligned} \tau & : S \times I \rightarrow \mathcal{E} + (S \times O \times A_X^*), \\ \text{emit}(\tau(s, i)) & \subseteq \text{cap}, \\ a \notin \text{cap} & \Rightarrow I_X(a) = \text{Reject}(a). \end{aligned}$$

3 Category and Effects

Definition 9 (Namespace object).

$$\begin{aligned} \nu & \in \{D, V, M, H, U, Q, W\}, \\ H_\nu & : X_\nu \rightarrow R, \\ (X_\nu, H_\nu) & \in \mathbf{Set}/R. \end{aligned}$$

Definition 10 (Placement functor).

$$\begin{aligned} \Omega_N & : \mathbf{Set}/R \rightarrow \mathbf{Set}/N, \\ (X, H) & \mapsto (X, \text{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \text{owner}_N \circ H_\nu : X_\nu \rightarrow N.$$

Proposition 2 (Placement functoriality).

$$f : X_\nu \rightarrow X_\mu, \quad H_\mu \circ f = H_\nu.$$

$$P_\mu^N \circ f = P_\nu^N.$$

Proof.

$$\text{owner}_N \circ H_\mu \circ f = \text{owner}_N \circ H_\nu.$$

□

Definition 11 (Writer effect monad).

$$\begin{aligned} A^* &= \text{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg f &= \text{let } f(x) = (y, \beta) \text{ in } (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^* \circ g^*) \circ f^* = h^* \circ (g^* \circ f^*).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

□

Definition 12 (Typed failure stack).

$$\mathsf{T}_A^\mathcal{E}(X) = \mathcal{E} + (X \times A^*).$$

$$\begin{aligned} \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \text{RecoverableReport} &\in A. \end{aligned}$$

Definition 13 (Protocol object).

$$\begin{aligned} \mathcal{P}_\nu &= (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu) \\ \tau_\nu : S_\nu \times I_\nu &\rightarrow \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

Definition 14 (Protocol morphism).

$$\begin{aligned} \Sigma : \mathcal{P}_\nu &\rightarrow \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*) \\ \sigma_A^* &: A_\nu^* \rightarrow A_\mu^*, \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu, \\ \mathsf{T}_{\sigma_A^*}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

4 Chord Overlay

$$\begin{aligned} \text{Base} &= \text{Chord}, \\ \text{Maintenance} &= \text{CorrectChord}, \\ \text{Object} &= \mathcal{R}_N, \\ \text{Witnesses} &= \{\text{Algorithms 1, 2, 3}\}. \end{aligned}$$

[1, 2]

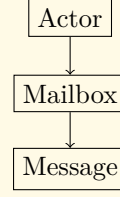


Figure 5: Actor Model

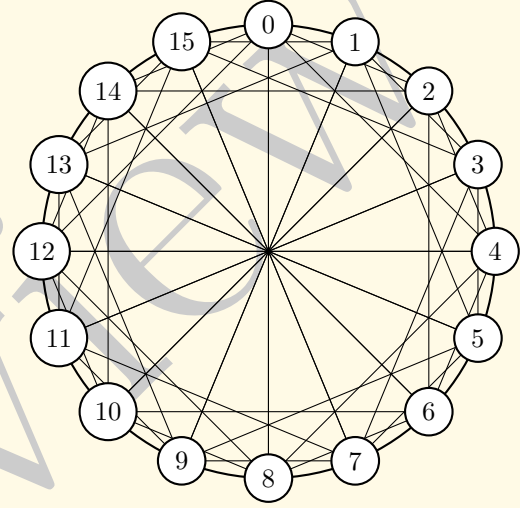


Figure 6: Chord algorithm, lookup protocol

Definition 15 (Local node state).

$$S_n = (p_n, L_n, F_n, E_n)$$

$$\begin{aligned} p_n &\in N \cup \{\perp\}, \\ L_n &\in N^{\leq r}, \\ F_n &: \mathbb{N} \rightarrow N, \\ E_n &\subseteq E, \\ p_n = \text{pred}_N(n), \quad L_n &= \text{Succ}_N^r(n). \end{aligned}$$

Definition 16 (Finger target).

$$\begin{aligned} \text{target}_i(n) &= n + 2^i, \\ \text{finger}_i(n) &= \text{owner}_N(\text{target}_i(n)), \\ F_n(i) &\in \{\perp, \text{finger}_i(n)\}. \end{aligned}$$

Algorithm 1 DHT Join Transitions

```
1: function BEGINJOIN( $n, known$ )
2:    $pred[n] \leftarrow none$ 
3:    $a \leftarrow FindSuccessor(n.did)$ 
4:   return Remote( $known, a$ )
5: function INSTALLSUCCESSOR( $n, s$ )
6:    $succ[n] \leftarrow take(r, [s])$ 
7:   return Remote( $s, QuerySuccList(s)$ )
8: function INSTALLSUCCLIST( $n, s, list$ )
9:    $succ[n] \leftarrow take(r, s :: list)$ 
10:  if head( $succ[n]$ )  $\neq none$  then
11:     $h \leftarrow head(succ[n])$ 
12:     $notify \leftarrow Notify(h, n.did)$ 
13:     $sync \leftarrow SyncStorage(h)$ 
14:    return Multi( $notify, sync$ )
15:  else
16:    return Noop
```

Invariant 2 (Ring fixpoint).

$$Inv_{ring}(N, S) \equiv \forall n \in N. p_n = pred_N(n) \\ \wedge L_n = Succ_N^r(n).$$

Invariant 3 (Lookup owner preservation).

$$Steady(N) \wedge Inv_{ring}(N, S) \\ \Rightarrow Lookup(S, k) \Downarrow owner_N(k). \\ \Downarrow = Reach_{AD}(2, 3).$$

5 DID and Resource Algebra

Definition 17 (DID namespace).

$$\mathcal{D} = (X_D, H_D, \Pi, Session), \quad H_D : X_D \rightarrow R. \\ D(k) = H_D(k), \\ owner_N(D(k)) \in N, \\ Session \subseteq X_D \times K \times Time.$$

Definition 18 (VID namespace).

$$\mathcal{V} = (X_V, H_V, Auth_V), \\ H_V : X_V \rightarrow R, \\ PrivateKey(H_V(x)) = \perp, \\ Auth_V(op, x, \sigma, caps) \in \{\top, \perp\}.$$

Algorithm 2 DHT Lookup Algorithm

```
1: function LOOKUP( $key, node$ )
2:    $s \leftarrow head(succ[node])$ 
3:   if  $s = none$  then
4:     return Repair( $QuerySuccList(node)$ )
5:   if  $key \in (node.did, s]$  then
6:     return Local( $s$ )
7:    $next \leftarrow closest\_preceding\_node(node, key)$ 
8:   if  $next = none$  then
9:     return Repair( $QuerySuccList(s)$ )
10:  else
11:     $a \leftarrow FindSuccessor(key)$ 
12:    return Remote( $next, a$ )
```

Algorithm 3 DHT Stabilization

```
1: procedure STABILIZE( $n, topo$ )
2:    $actions \leftarrow []$ 
3:    $c \leftarrow topo.pred$ 
4:    $s \leftarrow head(succ[n])$ 
5:   if  $c \neq none$  and  $c \in (n.did, s)$  then
6:      $succ[n] \leftarrow insert(succ[n], c)$ 
7:      $actions \leftarrow actions :: FindSuccessor(c)$ 
8:    $succ[n] \leftarrow take(r, merge(succ[n], topo.succ))$ 
9:   if head( $succ[n]$ )  $\neq none$  then
10:     $h \leftarrow head(succ[n])$ 
11:     $actions \leftarrow actions :: Notify(h, n.did)$ 
12:  return Multi( $actions$ )
```

Definition 19 (Biased order).

$$A \leq_C B \iff \delta_C(A) \leq \delta_C(B).$$

Definition 20 (Replicated entry algebra).

$$\begin{aligned} \mathcal{E}_v^{join} &= (E_v, \sqsubseteq, \sqcup, \perp), \\ \mathcal{E}_v^{final} &= (E_v, \text{conflict}_v, \text{finalize}_v). \\ x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \perp &= x, \\ \text{finalize}_v &: \text{conflict}_v(E_v) \rightarrow E_v. \\ \text{case}_1 &= \text{CRDT}. \end{aligned}$$

[3]

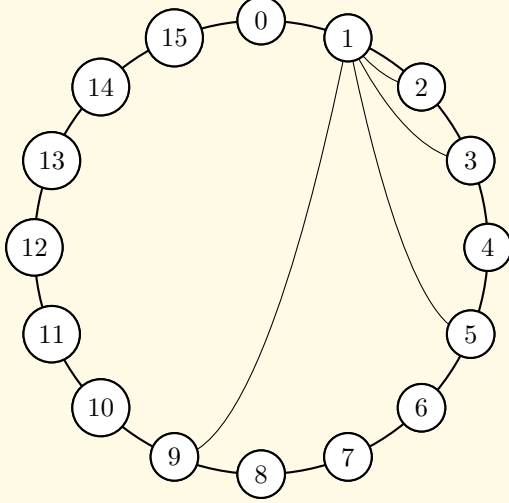


Figure 7: Chord algorithm, lookup protocol

Algorithm 4 Biased Circular Comparison

```

1: function BIASEDCOMPARE( $A, B, C$ )
2:    $a \leftarrow \delta_C(A)$ 
3:    $b \leftarrow \delta_C(B)$ 
4:   if  $a < b$  then
5:     return  $A <_C B$ 
6:   else if  $a > b$  then
7:     return  $B <_C A$ 
8:   else
9:     return  $A = B$ 

```

Invariant 4 (Placement).

$$\text{Inv}_{\text{place}} \equiv \forall \nu, x. \text{replicas}(H_\nu(x)) \subseteq \text{Rep}_N^r(H_\nu(x)).$$

Definition 21 (Mailbox).

$$\begin{aligned} \text{mbox}(d, e) &= H_M(\text{rings.mailbox} \| d \| e), \\ \text{entry}_M &= (\eta, \text{exp}, \text{proof}_s, \text{dst}, \text{cipher}). \end{aligned}$$

Definition 22 (Hidden service descriptor).

$$\begin{aligned} h &= (\nu, \rho, \text{Intro}, \text{Relay}, \\ &\quad \text{Cap}, K, \text{exp}, \text{rank}), \\ \text{VID}(h) &= H_H(h), \\ \text{Lookup}(h) &= P_H^N(h), \\ \text{Valid}(h, \sigma) &= \text{Auth}_H(h, \sigma). \end{aligned}$$

Definition 23 (Subring).

$$\mathcal{S}_u = (u, N_u, \text{owner}_{N_u}, \text{Succ}_{N_u}^{r_u}, E_u)$$

$$\begin{aligned} u &= H_U(\text{policy}), \\ \text{memberlog}_u &\in \text{MergeLog} \cup \text{FinalState}. \end{aligned}$$

6 Cryptographic Carriers and Traffic

Definition 24 (End-to-end carrier).

$$\begin{aligned} |\mathbb{G}| &= q, \\ \text{Scalar}(\mathbb{G}) &= \mathbb{Z}_q, \\ x &\in \mathbb{Z}_q, \\ h &= g^x, \\ \mathbb{G} &\neq R. \end{aligned}$$

Algorithm 5 ElGamal Encryption and Decryption

```

1: Input: Message  $m$ , private key  $x \in \mathbb{Z}_q$ , public
   key  $h = g^x$ , generator  $g$ 
2: Output: Encrypted message  $(c_1, c_2)$ 
3: procedure ENCRYPTION
4:   Choose a random integer  $k \in \mathbb{Z}_q$ 
5:   Compute  $c_1 = g^k$ 
6:   Compute  $c_2 = m \cdot h^k$ 
7:   return  $(c_1, c_2)$ 
8: procedure DECRYPTION
9:   Compute  $m = c_2 \cdot (c_1)^{-x}$ 
10:  return  $m$ 

```

Definition 25 (Signed encrypted frame).

$$f = (\text{sid}, \text{seq}, \text{last}, \text{cipher}, \text{pk}_s, \sigma_s).$$

$$m_f = \text{sid} \| \text{seq} \| \text{last} \| \text{cipher},$$

$$\begin{aligned} \text{FrameValid}(f) &\Leftrightarrow V(\text{pk}_s, \sigma_s, m_f) = \top \\ &\quad \wedge \text{Session}(D(\text{pk}_s), \text{sid}, e). \end{aligned}$$

Definition 26 (Secret sharing placement).

$$\begin{aligned} P &\in \mathbb{F}_q[X], \\ sh_i &= (i, P(i)) \in \mathbb{F}_q^2, \\ H_S(sh_i) &\in R, \\ \text{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.}) \end{aligned}$$

[4]

Definition 27 (Relay frame).

$$\begin{aligned}
r &= (\text{path}, \text{dst}, \nu, \text{session}, \text{payload}, \text{report}). \\
\text{RelayValid}(r) &= \text{SessionValid}(\text{session}) \\
&\quad \wedge \text{dst} \in R, \\
\text{SplitVocabulary} &= \text{MSRP}.
\end{aligned}$$

[5]

Algorithm 6 End-to-End Chunking Algorithm

```

1: procedure E2E CHUNKING
2:   Input: data, chunk size
3:   Output: chunks
4:   chunks  $\leftarrow \square$ 
5:   total-chunks  $\leftarrow \text{ceil}(\text{len}(\text{data})/\text{chunk-size})$ 
6:   for i in range(total-chunks) do
7:     chunk  $\leftarrow \text{data}[\text{i} * \text{chunk-size} : (\text{i}+1) * \text{chunk-size}]$ 
8:     append chunk to chunks
9:   return chunks

```

Invariant 5 (Chunk reassembly).

$$\begin{aligned}
\text{complete}(m) &\Leftrightarrow \forall i < \text{total}(m). \exists ! c_i \\
&\quad \wedge \sum_i |c_i| \leq B_m \\
&\quad \wedge \text{now} < \text{ttl}(m).
\end{aligned}$$

7 Ordering, Ranking, and Security

Definition 28 (Sidecar witness).

$$\begin{aligned}
w &= (\text{domain}, \text{root}, \text{height}, \text{time}, \text{finality}). \\
c &= H(\text{msg}), \\
\text{Witness}(\text{msg}, w) &\Leftrightarrow \text{Include}(c, w.\text{root}) = \top \\
&\quad \wedge \text{Final}(w) = \top.
\end{aligned}$$

Definition 29 (Witness order).

$$\begin{aligned}
a < b &\Leftrightarrow \text{domain}(w_a) = \text{domain}(w_b) \\
&\quad \wedge \text{time}(w_a) < \text{time}(w_b). \\
\chi &: \text{Domain}_a \times \text{Domain}_b \rightarrow \text{Order}.
\end{aligned}$$

Definition 30 (Ranking event).

$$q = (\text{subject}, \text{rater}, \text{behavior}, \text{evidence}, \text{epoch}, \text{weight}, \sigma).$$

$$\begin{aligned}
m_q &= \text{subject} \parallel \text{behavior} \parallel \text{evidence} \parallel \text{epoch}, \\
\text{RankValid}(q) &\Leftrightarrow V(k_{\text{rater}}, \sigma, m_q) = \top.
\end{aligned}$$

Definition 31 (Score aggregation).

$$\begin{aligned}
\text{score}_e(s) &= \text{robust}\{\text{weight}(q) \cdot \text{value}(q) : \\
&\quad q.\text{subject} = s\}.
\end{aligned}$$

$$\text{robust} \in \{\text{median}, \text{trimmedMean}, \text{declared}\}.$$

Attack	Predicate	Rings control
Sybil	many DIDs, one actor	admission cost, ranking, challenge
Replay	stale valid σ	nonce, TTL, session epoch
MITM	wrong key for DID	DID proof, signed envelope, E2E
DoS	unbounded resource use	quotas, ranking, chunk bounds

Table 2: Security predicates and controls

Obligation 2 (Replay rejection).

$$\begin{aligned}
\text{accept}(s, \eta, t) &\Rightarrow \eta \notin \text{Seen}_s \wedge t < \text{exp}(s), \\
\text{Seen}'_s &= \text{Seen}_s \cup \{\eta\}, \\
\text{emitDelivery} &< \text{Seen}'_s.
\end{aligned}$$

8 Verification and Evaluation

Definition 32 (State relation).

$$C = (S, Q)$$

$$\frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau(S, i) = (S', O, \beta)}{(S, Q) \rightarrow (S', Q_0 \cdot \beta)}$$

$$\text{Explore} = \text{TLAStyle}(\rightarrow)$$

[6, 7]

Invariant 6 (Effect refinement).

$$\text{Impl}_A(s, i) \preceq I_A^\sharp(\tau(s, i)).$$

$$\pi_{state}(\text{Impl}_A) = \pi_{state}(I_A^\sharp \circ \tau).$$

Obligation 3 (Topology coverage).

$$\mathcal{L} = \{\text{line}, \text{wrap}, \text{gap}, \text{partition}, \text{merge}, \text{churn}\}$$

$$\mathcal{S} = \{\text{loss}, \text{dup}, \text{reorder}, \text{timeout}, \text{restart}\}.$$

$$\text{CheckCase} = (L, S, P), \quad L \in \mathcal{L}, \quad S \in \mathcal{S}.$$

Definition 33 (Latency decomposition).

$$T_{total} = T_{conn} + h \cdot T_{hop} + T_{crypto} + T_{app} + T_{witness}.$$

$$h = O(\log |N|) \quad (F_n \text{ populated}).$$

Fraction of failed nodes	Mean routing hops	Mean num. of lookup timeouts
0	3.84	0.0
0.1	4.03	0.60
0.2	4.22	1.17

Table 3: Latency on failed node rate of Chord [2]

Definition 34 (Reliability baseline).

$$\begin{aligned} \text{failrate} &= \frac{|F|}{|Q|}, \\ \text{timeoutMean} &= \frac{1}{|Q|} \sum_{q \in Q} \text{to}(q). \end{aligned}$$

Obligation 4 (Evaluation morphism).

$$\begin{aligned} \Phi_{eval} : (h, \text{timeouts}, \text{failures})_{\text{Chord}} \\ \mapsto (T_{total}, \text{timeoutMean}, \text{failrate})_{\text{Rings}}, \end{aligned}$$

$$\Delta\Phi = T_{conn} + T_{crypto} + T_{witness} + T_{app}.$$

9 Related Work

10 Conclusion

$$\mathcal{R} \setminus \setminus \setminus f = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathcal{T}_A^\mathcal{E}, \Pi, \mathbb{G}, \mathcal{X}).$$

join/leave rate (per sec./stab. period)	Mean routing hops	Mean num. of lookup timeouts	Lookup failures per 10k
0.05/1.5	3.90	0.5	0
0.10/2	3.83	0.11	0
0.15/4.5	3.84	0.16	2
0.20/6	3.81	0.23	5
0.25/7.5	3.83	0.30	6
0.30/9	3.91	0.34	8
0.35/10.5	3.94	0.42	16
0.40/12	4.06	0.46	15

Table 4: Failure rate of Chord [2]

System	Metric	Ownership law
Kademlia	XOR	bucket routing
fam.		
Relay	path	no structured
mixnets		owner
Rings	circular	owner, Succ, Rep

Table 5: Routing and ownership comparison [8–12]

$$\begin{aligned} \text{Overlay} &= \text{CorrectChord}(R, N, r), \\ \text{Resource} &= (X_\nu, H_\nu) \in \mathbf{Set}/R, \\ \text{Placement} &= \Omega_N(X_\nu, H_\nu), \\ \text{Transition} &= \tau_\nu : S_\nu \times I_\nu \rightarrow Y_\nu, \\ Y_\nu &= \mathcal{T}_{A_\nu}^\mathcal{E}(S_\nu \times O_\nu), \\ \text{Crypto} &= \mathbb{G} \vee \mathbb{F}_q, \quad \text{Crypto} \cap R = \emptyset, \\ \text{Evidence} &= \{\text{figures, tables, algorithms}\} \\ &\quad \cup \{\text{invariants, obligations}\}. \end{aligned}$$

References

- [1] Pamela Zave. Reasoning about identifier spaces: How to make chord correct. *IEEE Transactions on Software Engineering*, 43(12):1144–1156, 2017. <https://arxiv.org/abs/1610.01140>.
- [2] I. Stoica, R. Morris, and D. Karger. Chord: A scalable peer-to-peer lookup service for internet applications. <https://dl.acm.org/doi/10.1145/383059.383071>, 2001.

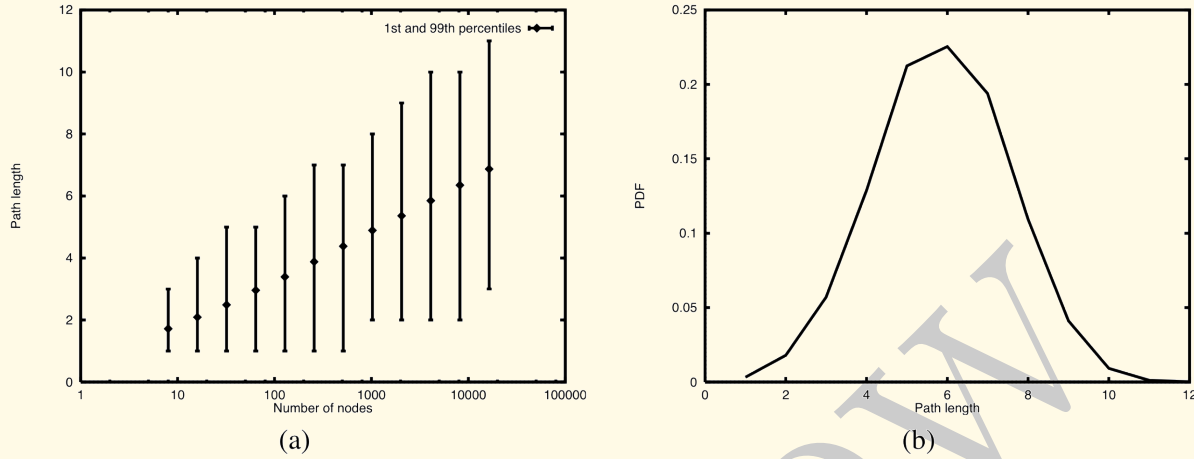


Figure 8: Chord lookup path [2]

- [3] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011.
- [4] A. Shamir. How to share a secret. <https://link.springer.com/article/10.1007/BF00185039>, 1979.
- [5] B. Campbell, R. Mahy, and C. Jennings. The message session relay protocol (msrp). RFC 4975, 2007. <https://www.rfc-editor.org/info/rfc4975>.
- [6] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [7] Jonathan Nadal. Stateright: Model checking for implementing distributed systems, 2026. <https://www.stateright.rs/>.
- [8] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the xor metric. *International Workshop on Peer-to-Peer Systems*, pages 53–65, 2002.
- [9] Juan Benet. Interplanetary file system: A p2p file system for the next web. *Proceedings of the 24th International Conference on World Wide Web*, pages 1149–1160, 2015.
- [10] J. R. Etheridge, M. Castro, and I. Stoica. Libp2p: A modular network stack for p2p systems. *USENIX Association Conference on Networked Systems Design and Implementation*, 14:1–15, 2017.
- [11] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. *ACM Transactions on Computer Systems*, 22(3):303–327, 2004.
- [12] Nym Development Team. Nym: A decentralized mix network. *Journal of Privacy and Security*, 6(2):135–144, 2021.